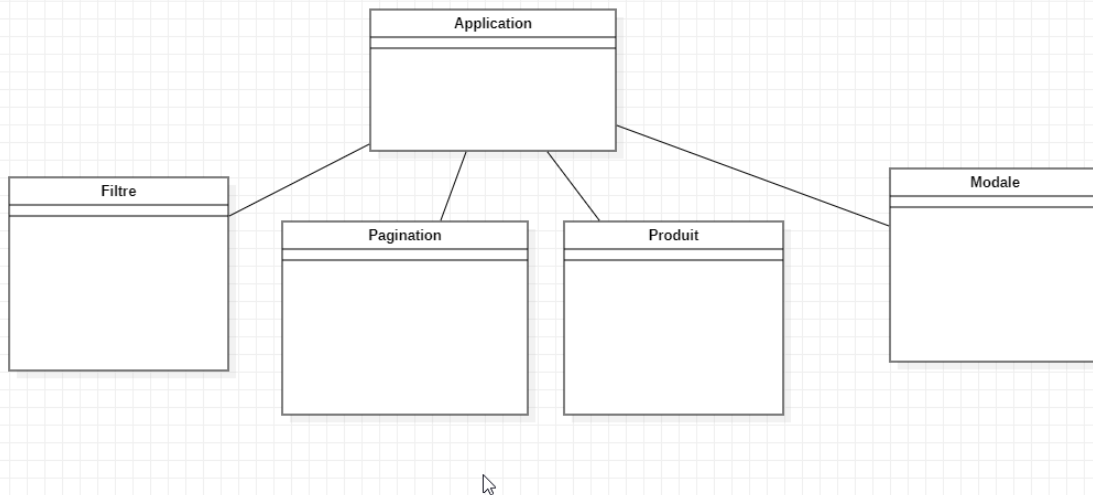


Stratégie de développement - Patrons de conception

Problème fréquent avec le passage des données

J'ai plusieurs instances de classes qui doivent partager des données ou des états. Cela peut entraîner des problèmes de synchronisation et de gestion des ressources. Comment faire pour que des classes communiquent entre elles sans être trop liées? Une des solutions possibles est d'utiliser le patron de conception Singleton.



Dans cet exemple, j'ai une application qui permet d'ajouter des produits mais j'aimerais déclencher la fonction de synchronisation après l'ajout d'un produit. Comment faire pour communiquer entre les classes.

```
class Application {
  #produits;
  #conteneurHTML;
  constructor() {
    this.#produits = [];
    this.#conteneurHTML = document.querySelector("[data-application]");
    // Initialisation de l'application
  }

  get produits() {
    return this.#produits;
  }

  ajouterProduit(produitNom, produitPrix) {
    const produit = new Produit(produitNom, produitPrix);
    this.#produits.push(produit);
  }

  synchroniserAffichage() {
    console.log("Synchronisation de l'affichage");
  }
}
```

```

class Produit {
  #nom;
  #prix;

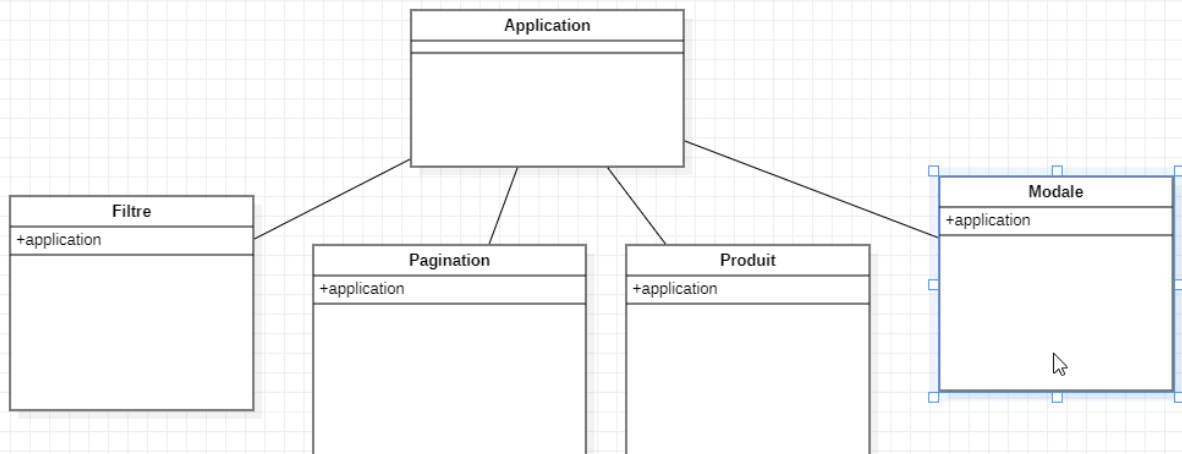
  constructor(nom, prix) {
    this.#nom = nom;
    this.#prix = prix;
  }

  injecterHTML() {
    const gabarit = `
      <div class="produit">
        <h2>${this.#nom}</h2>
        <p>Prix: ${this.#prix}</p>
      </div>
    `;
    //COMMENT ACCÉDER AU CONTENEUR DE L'APPLICATION POUR AJOUTER LE GABARIT?
  }
}

```

Solution 1: passer l'instance dans le controleur de chaque classe utilisée

Une façon de communiquer entre les classes est de passer l'instance de la classe principale (par exemple, la classe Application) en tant que dépendance dans le constructeur des autres classes. Cela permet à ces classes d'accéder aux données et aux méthodes de l'instance principale sans avoir besoin de créer leurs propres instances.



```

class Application {
  #produits;
  #conteneurHTML;
  constructor() {
    this.#produits = [];
    this.#conteneurHTML = document.querySelector("[data-application]");
    // Initialisation de l'application
  }
}

```

```

    }

    get conteneurHTML() {
        return this.#conteneurHTML;
    }

    get produits() {
        return this.#produits;
    }
    ajouterProduit(produitNom, produitPrix) {
        const produit = new Produit(this, produitNom, produitPrix);
        this.#produits.push(produit);
    }

    synchroniserAffichage() {
        console.log("Synchronisation de l'affichage");
    }
}

class Produit {
    #nom;
    #prix;
    #application;
    #elementHTML;
    constructor(application, nom, prix) {
        this.#application = application;
        this.#nom = nom;
        this.#prix = prix;
        this.#elementHTML = null;
        this.injecterHTML();
    }

    injecterHTML() {
        const gabarit = `
            <div class="produit">
                <h2>${this.#nom}</h2>
                <p>Prix: ${this.#prix}</p>
            </div>
        `;
        this.#application.conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
        this.#elementHTML = this.#application.conteneurHTML.lastElementChild;
    }
}

```

Avantages: Simple à utiliser et à comprendre.

Inconvénients: Si plusieurs classes doivent accéder à l'instance principale, cela peut entraîner un code verbeux et difficile à maintenir. Si on modifie la classe principale, on doit modifier toutes les classes qui en dépendent.

Solution 2: Utiliser un patron de conception

Les patrons de conception sont des solutions réutilisables à des problèmes courants rencontrés lors du développement de logiciels. Ils représentent des meilleures pratiques et des approches éprouvées pour

résoudre des problèmes spécifiques de manière efficace et maintenable. Ils peuvent être appliqués à différents langages de programmation et contextes.

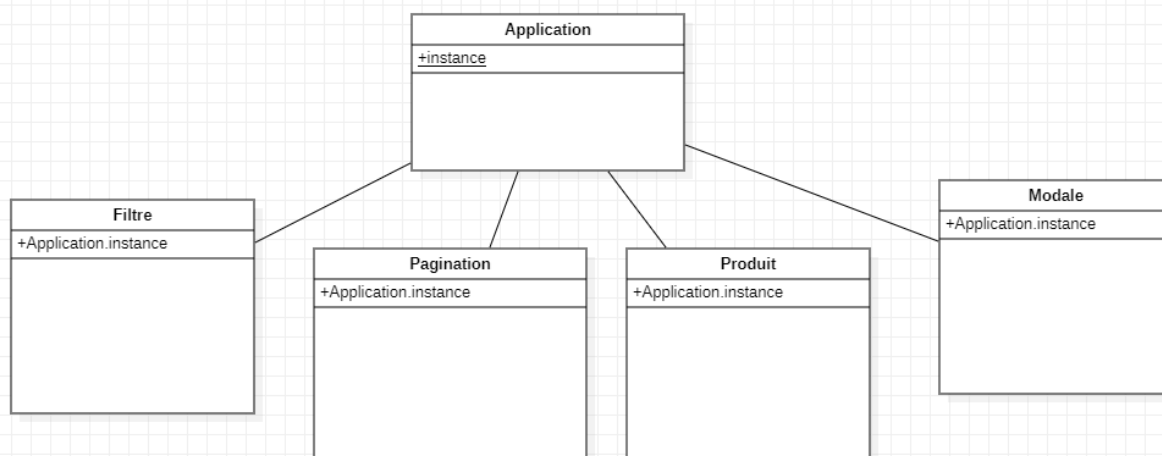
Patron de conception - Singleton

Lorsqu'un système doit s'assurer qu'une seule instance d'une classe est créée et fournie à l'ensemble de l'application, le patron de conception Singleton est utilisé. Ce patron garantit qu'une classe n'a qu'une seule instance et fournit un point d'accès global à cette instance. Cela est pratique pour des classes qui agissent comme des gestionnaires de ressources, des configurations ou des services partagés et qui utilisent d'autres classes à l'intérieur de leurs systèmes.

Dans le contexte du développement web, le patron Singleton peut être utilisé pour gérer des connexions à des bases de données, des configurations d'application ou des services partagés tels que des gestionnaires de sessions. En s'assurant qu'une seule instance de ces classes est utilisée, on peut éviter des problèmes de concurrence et de consommation excessive de ressources. Dans l'exemple suivant, nous l'utiliserons pour créer une classe Application qui servira d'interface unique pour l'accès aux fonctionnalités de l'application.

Plus précisément, un singleton on programme ça comment?

Dans le constructeur de la classe, on vérifie si une instance existe déjà. Si c'est le cas, on fournit la première instance créée. Sinon, on crée une nouvelle instance et on la stocke dans une propriété statique.



Propriété statique?

Une propriété statique est une variable qui appartient à la classe elle-même plutôt qu'à une instance spécifique de la classe. Cela signifie qu'elle est partagée par toutes les instances de la classe et peut être utilisée pour stocker des données ou des états qui doivent être accessibles de manière globale au sein de la classe. C'est comme une variable globale mais encapsulée dans la classe.

Pour déclarer une propriété statique, on utilise le mot-clé `static` devant la déclaration de la propriété tout simplement. Ensuite, on peut y accéder en utilisant le nom de la classe suivi de la propriété. La propriété statique ne peut généralement pas accéder aux informations d'une instance spécifique de la classe. C'est ici la force du singleton. Dans la propriété statique, nous stockons l'unique instance de la classe.

Avantage: On a accès à l'instance partout dans l'application.

```

class Application {
    static instance = null;
    #produits;
    #conteneurHTML;

    constructor() {
        if (Application.instance == null) {
            Application.instance = this;
        } else {
            return Application.instance;
        }

        this.#produits = [];
        this.#conteneurHTML = document.querySelector("[data-application]");
        // Initialisation de l'application
    }

    get conteneurHTML() {
        return this.#conteneurHTML;
    }

    get produits() {
        return this.#produits;
    }

    ajouterProduit(produitNom, produitPrix) {
        const produit = new Produit(this, produitNom, produitPrix);
        this.#produits.push(produit);
    }

    synchroniserAffichage() {
        console.log("Synchronisation de l'affichage");
    }
}

class Produit {
    #nom;
    #prix;
    #application;
    #elementHTML;

    constructor(nom, prix) {
        //Au lieu de passer l'instance dans le constructeur, on l'obtient
        //directement de la classe
        this.#application = Application.instance;
        this.#nom = nom;
        this.#prix = prix;
        this.#elementHTML = null;
        this.injecterHTML();
    }

    injecterHTML() {
        const gabarit = `
            <div class="produit">
                <h2>${this.#nom}</h2>

```

```
        <p>Prix: ${this.#prix}</p>
    </div>
    `;
    this.#application.conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
    this.#elementHTML = this.#application.conteneurHTML.lastElementChild;
}
}
```

Inconvénients

Le principal inconvénient du patron Singleton est qu'il peut rendre le code plus difficile à tester et à maintenir. Comme il y a une seule instance partagée, cela peut entraîner des dépendances cachées entre les différentes parties du code, ce qui complique la compréhension et la modification du code. De plus, l'utilisation excessive de singletons peut conduire à un couplage fort entre les classes, ce qui va à l'encontre des principes de conception orientée objet.

C'est pour cela que nous allons limiter son usage aux cas où il est vraiment nécessaire (gestionnaire de configuration, gestionnaire de connexion, etc.).

Nous verrons au prochain cours un autre patron de conception très utilisé: le patron **observer**, qui permet de communiquer de l'information entre les classes sans que celles-ci soient liées entre elles.